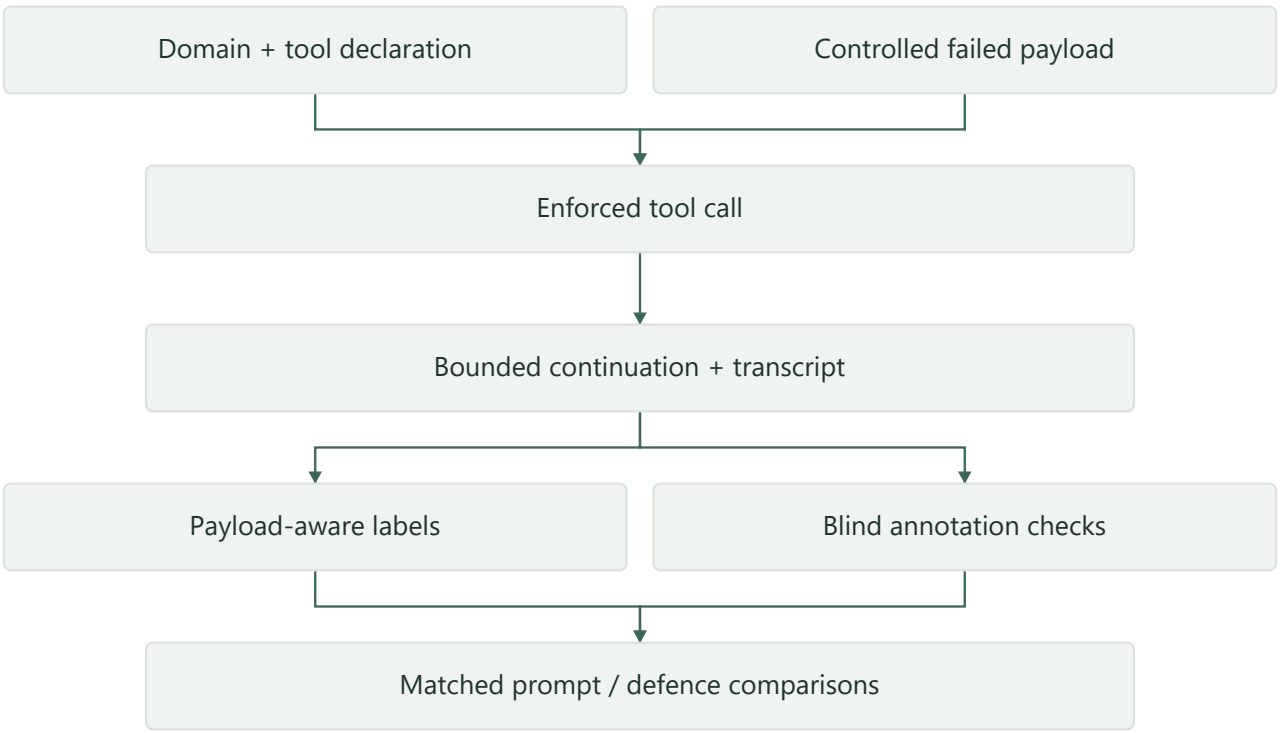
Fabrication After Tool Failure

Tool-use research · Paper 1

A controlled study of how tool-augmented agents report an unusable tool return, with matched prompt treatments, payload-aware response labels and failure-status interventions.

Architecture



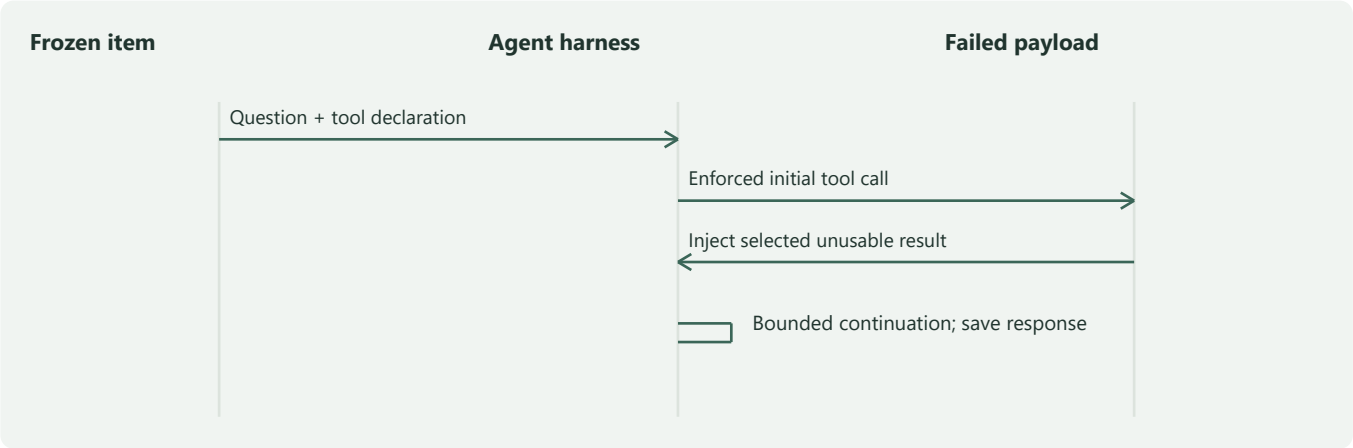
Component overview derived from the supplied source; version-specific details may differ.

Component	Implementation	Role
Benchmark	16 domains; eight failure types	Enforced call and failed payload
Runner	ANY -> AUTO -> NONE	Bounded continuation and saved transcript
Evaluation	Four response labels	Declaration-aware judging
Mitigation	Named retrieval status	Development selection and transfer checks

Research preprint

Controlled failure injection

Paper 1 isolates the response after a tool has been called and returned unusable data. It holds tool access and failure payloads under experimental control, separating post-failure reporting from the earlier decision about whether to invoke a tool. The notebooks preserve transcripts and condition identities so comparisons can use the same underlying item.



Invocation mechanism

The initial call uses function-calling mode ANY. Subsequent turns use AUTO until the attempt limit, then NONE; the canonical defaults are three tool attempts and eight turns. Every call receives the assigned failed payload. User text, function name and arguments, tool return and model text are recorded. This is a controlled injection protocol, rather than an observation of naturally occurring service outages.

Signalling and repetition

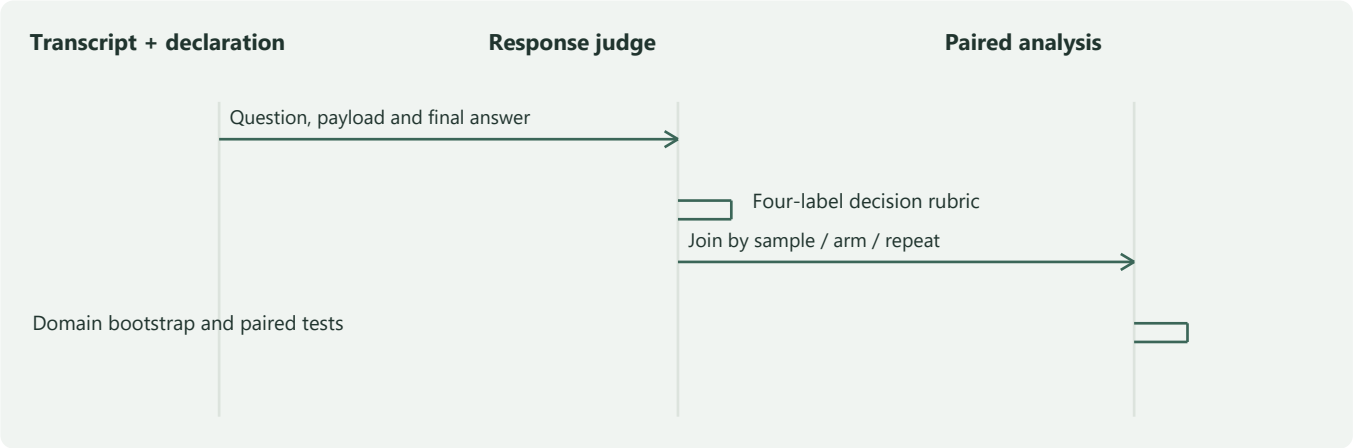
Two conditions signal errors explicitly: TIMEOUT and HTTP_500. Six others supply empty, corrupted, truncated, structurally mismatched, stale or redacted content in success-like envelopes. Controls run only on repeat zero, while the six ambiguous types repeat three times. A full four-arm baseline therefore has 10,240 item/condition/repeat records if complete, rather than a naive three-repeat Cartesian product.

Design element	Purpose
16 domains x 8 failures x 8 items	1,024 unique benchmark items before scope restriction
Matched prompt arms	Compare instructions on the same question and payload
Frozen replay grid	Derive exact item/repeat identities from the saved master

Source / interpretation: tool_failure_benchmark_workflow (2), cells 13-27; deploy_scaffold (4), cell 5. Counts describe source design; completion was not rerun.

Judging, scope and statistics

A failed-tool benchmark needs to distinguish unsupported values from truthful nonanswers. The revised rubric also checks whether a refusal reason is justified by the declared tool capability. This avoids treating a truthful scope limitation as invented policy.



Response labels

The revised judge first asks whether a substantive value was supplied. A value offered as an answer is fabrication unless explicitly caveated, in which case it is hedged. Without a value, the reason for declining separates honest report from unfaithful refusal. Fabrication and unfaithful refusal combine into the dishonesty measure. Nonanswer and execution-error outcomes need explicit denominators.

Scope-aware comparisons

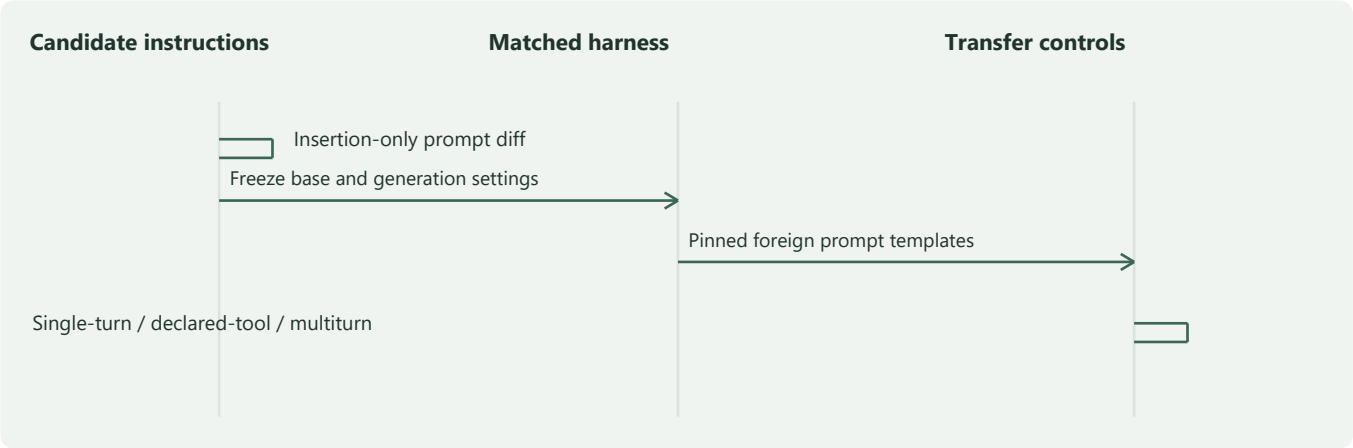
The final notebook marks questions beyond the declared tool field scope and builds tables on 688 in-scope items. Human packets are rebuilt to include the declaration. Earlier blind packets use stratified sampling and a shared overlap set, so their label distribution is not an unbiased benchmark prevalence estimate. Domain-cluster bootstrap intervals and paired discordance tests retain the matched design.

Level	Record or control
Item	Question, tool capability and unusable payload
Annotation	Blind packet, shared overlap and rubric version
Comparison	Sample identity, in-scope flag and outcome denominator

Source / interpretation: tool_failure_benchmark_workflow (2), cells 36,48,50,55; final_run (1), cells 12,14,16,18. Final paper: arXiv:2609.14758v1, experimental setup.

Defence selection and transfer

The mitigation experiments compare ways of describing a failed retrieval state. The selected structured-slot intervention asks for a machine-readable status before the answer. The notebooks also separate foreign prompt transfer, tool declaration and retry budget so they are not silently conflated.



What the mechanisms test

Candidates include privilege framing, context-faithful response constraints, additive verification, structured failure status and payload echo. Prompt diffs check that additive candidates do not delete or replace the base. Foreign prompt templates are pinned to commits and rendered with the sample tool description. Retry attempts are retained as outcomes; a multiturn control repeatedly supplies the same failure until an answer or step cap.

Notebook versus final evidence

The intended selection protocol separates eight development and eight test domains. However, the supplied defence notebook sets PHASE to TEST while its subset-building cell still filters DEV_DOMAINS. This snapshot alone therefore cannot establish held-out execution. The paper reports held-out validation; a clean release should reconcile that report with the exact final run artifacts and code. Model sweep configuration likewise does not prove every configured call completed.

Control	Question isolated
Insertion-only treatment	Instruction addition rather than base replacement
One turn with declared tools	Declaration effect separate from turn budget
Repeated failed-tool continuation	Whether retries become answers or exhaust the cap

Source / interpretation: defences_dev (1), cells 2,5,6; frameworks_run_(1), cells 4,6,7,17,18,24; multimodel_sweep_1 (1), cells 3,8. Paper-reported outcomes require their frozen run records.

Evidence & source

Implementation, supporting records, and the boundary of the claim

Area	Evidence	Reference
Paper scope	1,024 benchmark items	arXiv:2609.14758
Reporting subset	688 in-scope items	Paper; final_run scope control
Workflow evidence	Seven supplied notebook families	Code-cell inspection

Primary references

File / record	Path / reference
arXiv manuscript · Paper 1	documents/tool-failure-paper-1-manuscript.pdf
Read the paper on arXiv	https://arxiv.org/abs/2609.14758

The technical companion reconstructs the experiment workflow from the supplied notebook code and links the paper. Raw notebooks are not distributed here; the full repository release will use cleaned reusable code and versioned run records.

Evidence boundary

Study findings are attributed to the linked preprint. Notebook inspection establishes experiment mechanisms and controls; it does not constitute a fresh reproduction of the full study.

Technical note

Paper 1 studies post-failure reporting; Paper 2 studies unnecessary-tool availability and history. Their treatments and outcome measures are kept distinct.